RFC 9700 and RFC 8555 compliance audit: security hardening checklist, PKCE mandates, and TLS configurations for the Semester system

Computer
Semester system risk assessment: automated compliance monitor for RFC 9700, ACME protocols, and PKCE vulnerabilities
Computer
Draft the RFC 9700 compliance checklist for OIDC
Write the wildcard TLS and custom-domain routing spec
Detail the phased security hardening milestones for Semester

Semester needs an evidence-based security control system—not a “compliant” badge inferred from configuration. The deliverable should combine an RFC 9700/OIDC control checklist, verified wildcard TLS and custom-domain lifecycle, and an automated monitor that records what was tested, where, when, and with what result.

The current assessment is **unverified**: this conversation contains architecture proposals and dated project evidence, but no live identity configuration, certificate inventory, or executed security-test results. Nothing below certifies Semester as compliant.

1. Audit scope and standards interpretation

Keep four evaluations separate:

Evaluation	What it establishes
RFC 9700 assessment	OAuth security best-practice adherence for applicable clients and authorization/resource servers
OIDC assessment	Correct authentication, token validation, discovery, and identity mapping
RFC 8555 assessment	Correct ACME behavior for components Semester actually operates
Semester security assessment	Tenant isolation, recovery, domain ownership, support controls, privacy, and operational resilience

RFC 9700 is an OAuth Security Best Current Practice; it is not a complete OIDC implementation specification. RFC 8555 defines ACME, not a general TLS cipher-suite baseline or a comprehensive domain-security program. [1] [2]

If Vercel or another provider operates ACME, Semester should audit its domain configuration, provider assurance, issuance/renewal outcomes, and failure handling. Semester should not claim to have directly tested that provider’s internal ACME implementation.

2. RFC 9700 and OIDC compliance checklist

In the table, “Semester policy” means an intentional product requirement that may be stricter than the underlying standard.

Authorization and token flows

ID	Control	Requirement and test	Evidence required
AUTH-01	Authorization Code flow	Interactive clients use code flow; implicit flow disabled	Client inventory and rejected implicit-flow test
AUTH-02	PKCE	Public clients require PKCE; Semester requires it for confidential clients too	Missing-PKCE rejection for every applicable client
AUTH-03	`S256` only	Semester policy rejects `plain` and unknown methods	Negative tests and discovery configuration
AUTH-04	PKCE binding	Code is bound to the original challenge, client, and redirect URI	Wrong-verifier and cross-client redemption tests
AUTH-05	Downgrade prevention	A verifier cannot compensate for a challenge missing from the original request	Downgrade test
AUTH-06	Exact redirects	Match registered redirect URIs exactly, except permitted native loopback-port handling	Redirect-mutation test corpus
AUTH-07	No open redirects	Callback and continuation routes cannot forward credentials to arbitrary destinations	Open-redirect tests
AUTH-08	Transaction correlation	State/nonce checks follow the selected flow; Semester requires transaction-bound values	Missing, altered, and replayed-value tests
AUTH-09	Mix-up defense	Bind the expected authorization server to each transaction	Wrong-school/wrong-issuer callback tests
AUTH-10	One-time codes	Reject reused, expired, or incorrectly bound authorization codes	Code-replay tests
AUTH-11	No password grant	Resource Owner Password Credentials grant unavailable	Rejected grant request
AUTH-12	Refresh-token protection	Public-client refresh tokens rotate or are sender-constrained	Replay/reuse and revocation tests
AUTH-13	Token restriction	APIs validate intended audience and necessary scopes	Wrong-audience and excess-scope tests
AUTH-14	Client authentication	Confidential clients use supported, secure authentication; prefer asymmetric methods where feasible	Client registrations and authentication tests

RFC 9700 mandates PKCE for public clients, recommends it for confidential clients, requires exact redirect matching subject to its native-app exception, and prohibits the password grant. It also addresses mix-up, token replay, and refresh-token protections. [1]

PKCE is not client authentication. Confidential clients still need their approved client-authentication mechanism; public clients must not rely on an embedded secret as proof of identity. [3]

OIDC, federation, and account controls

ID	Control	Semester acceptance criterion
OIDC-01	Signature validation	Reject invalid signatures, unsupported algorithms, and inappropriate keys
OIDC-02	Claim validation	Validate issuer, audience, expiration, nonce where applicable, and authorized-party handling when required
OIDC-03	Token purpose	ID tokens cannot be used as general API access tokens
OIDC-04	Trusted discovery	Federation metadata comes only from approved connection configuration
OIDC-05	Key rotation	Valid overlap works; retired or untrusted keys fail under documented policy
FED-01	External identity mapping	Trusted connection + issuer + subject maps to the immutable account
FED-02	Safe linking	Linking requires proof of control and an explicit user-approved workflow
FED-03	Email collisions	Matching email strings never trigger automatic account merging
FED-04	Membership authority	Authentication alone does not establish enrollment, role, or administrative access
FED-05	Scoped offboarding	Ending one school relationship does not remove unrelated relationships
SESSION-01	Browser sessions	Host-scoped secure cookies, CSRF defenses, and server-held tokens where applicable
SESSION-02	Revocation	Membership/session changes invalidate access within a tested, documented bound
RECOVERY-01	Recovery	Recovery cannot bypass school policy or depend solely on the mailbox being recovered

OIDC supplies the issuer/subject identity model and authentication-response validation requirements. These controls must be tested alongside—not substituted by—the OAuth checklist. [4]

Required test matrix

Run applicable tests against:

- First-party web clients.
- Native clients.
- Each school-federation protocol/configuration class.
- Custom-domain callbacks.
- Approved partner clients.
- Each API audience.
- Recovery, linking, and offboarding workflows.

Passing one default client does not prove all clients are secure.

3. Wildcard TLS and custom-domain routing specification

3.1 Separate hostname classes

Use illustrative domain `D`:

```
identity.D                  Stable central OIDC issuer
app.D                       First-party application
mail.D                      Mail interface
school-slug.D               Semester tenant hostname
learn.school.example        Institution-owned custom hostname
```

Prefer a separately controlled namespace for tenant-published content if institutions or users can publish active content. Shared identity does not require shared parent-domain cookies.

3.2 Wildcard certificate boundaries

Name	Covered by `*.D`?
`app.D`	Yes
`school-slug.D`	Yes
`D`	No; include the apex separately if needed
`portal.school-slug.D`	No
`learn.school.example`	No; needs its own certificate arrangement

Vercel documents wildcard issuance through DNS validation and distinguishes first-level wildcard hosts from deeper names. Its current documentation also supports delegated `_acme-challenge` validation where moving the entire DNS zone is unsuitable.[5] [6]

A wildcard certificate does not authorize wildcard OAuth callbacks.

3.3 Domain registry

Proposed logical schema:

```
domain_binding
  id
  normalized_hostname
  tenant_id or service_id
  class
  ownership_verification_state
  routing_state
  certificate_state
  provider_reference
  configuration_version
  approval_reference
  verified_at
  last_checked_at
```

```
  suspended_at
  removed_at
```

Constraints:

- Globally unique active hostname binding.
- Explicit tenant/service ownership.
- Reserved first-party hostnames cannot be claimed by tenants.
- Unknown or inactive hosts fail closed.
- Reassignment requires fresh verification.
- Historical bindings remain auditable.

3.4 Provisioning workflow

```
Request
→ verify requester's institutional authority
→ normalize hostname and check conflicts
→ create pending binding
→ issue expiring ownership challenge
→ verify DNS ownership
→ create hosting-provider binding
→ verify routing DNS
→ issue certificate through provider/ACME
→ test TLS, host routing, and tenant isolation
→ register exact approved OAuth callback
→ approve activation
→ enable traffic
```

Keep these proofs separate:

1. Requester authority.
2. DNS control.
3. Certificate issuance.
4. Correct routing.
5. Correct tenant authorization.

A certificate does not prove that the hostname is mapped to the right institution.

3.5 Vercel wildcard configuration

If the verified deployment uses Vercel:

1. Add the intended wildcard domain to the correct production project.
2. Choose supported DNS management or certificate-validation delegation.
3. Retrieve the exact required DNS records from the provider.
4. Configure wildcard traffic routing separately from validation delegation.

5. Confirm certificate issuance and renewal access.
6. Activate only registry-approved hostnames.

For external DNS, Vercel documents delegation of the relevant `_acme-challenge` subdomain and a wildcard routing CNAME. Do not invent provider record values or assume a DNS CNAME and an NS delegation are interchangeable.[5]

For `*.example.com`, the validation name is `_acme-challenge.example.com`; for `*.preview.example.com`, it is `_acme-challenge.preview.example.com`. Delegation may affect other issuers that need the same challenge name, so coordinate with institutional IT.[6]

3.6 TLS baseline

Proposed Semester policy:

- TLS 1.2 and 1.3 support; disable older versions where configurable.
- Provider-approved modern cryptographic configuration.
- Valid hostname, trusted chain, and appropriate certificate usage.
- No credentials, authorization codes, or tokens transmitted over HTTP.
- HTTPS redirects without sensitive query-string logging.
- HSTS introduced after all affected hosts are verified.
- Do not enable HSTS `includeSubDomains` or preload until the entire relevant namespace is ready.
- Certificate-expiry, DNS-drift, and renewal-failure monitoring.

These are Semester operational requirements, not claims that RFC 8555 mandates this complete baseline.

3.7 Runtime routing

```
Trusted proxy receives request
→ validate and normalize hostname
→ look up active binding
→ derive candidate tenant context server-side
→ authenticate
→ verify active membership
→ enforce resource policy
→ return permitted result
```

Reject arbitrary forwarded-host values. Scope caches by tenant, resource, and authorization context as appropriate so cached content cannot leak across domains.

Use host-only cookies. A custom school host authenticates through the central issuer and receives its own application session—not a cookie shared with every Semester subdomain.

3.8 Removal workflow

```
Suspend routing
→ disable callback registrations
→ invalidate domain/session caches as appropriate
→ remove provider binding
→ notify owner to remove stale DNS
→ retain audit tombstone
→ require fresh proof before reuse
```

Do not delete institutional records because a hostname is removed.

4. Automated security and compliance monitor

Architecture

```
Approved inventories
  Clients · Federation connections · Domains · APIs
                          |
             Read-only configuration collectors
                          |
     Passive observations + controlled synthetic tests
                          |
                Control evaluation engine
                          |
              Evidence store and findings queue
                          |
     Owner assignment · Remediation · Retest · Release gate
```

The monitor should report control status, not award certification.

Finding states

Use:

```
Pass
Fail
Not tested
Not applicable — reason required
Blocked — missing access or evidence
Exception — approved, owned, expiring
```

Never turn “configuration says enabled” into “runtime test passed.”

Proposed evidence record

```
{
  "control_id": "AUTH-05",
  "environment": "staging",
  "subject_type": "oauth_client",
  "subject_reference": "<client-reference>",
  "configuration_version": "<version>",
  "result": "not_tested",
  "evidence_reference": null,
  "checked_at": null,
  "owner": "<identity-owner>",
  "exception_reference": null
}
```

Evidence should distinguish:

- Configuration inspection.
- Passive observation.
- Executed negative test.
- Operational drill.
- Provider documentation/assurance.
- Institution approval.

Monitoring schedule

Proposed starting schedule—not a measured SLA:

Trigger	Evaluation
Pull request	Configuration linting, redirect changes, secrets checks
Deployment	Discovery, token-validation, PKCE, audience, and routing probes
Domain change	Ownership, binding, TLS, callback, and tenant-isolation checks
Daily	Certificate/DNS health and inventory drift
Periodic staging run	Replay, downgrade, linking, revocation, and federation tests
Release gate	Evidence freshness, unresolved findings, exceptions
Operational exercise	Key rotation, recovery, domain removal, restore, incident response

Safe monitoring limits

- Use test accounts, dedicated test tenants, and synthetic messages.
- No exploitation, role changes, account linking, or certificate revocation against real users without approval.
- No production certificate-renewal storms.

- No scanning unrelated institutional hosts.
- No raw tokens, secrets, or protected student records in findings.
- Keep scanners isolated from powerful DNS and identity-management credentials.
- Require human approval for fixes that change external state.

The monitor can detect a missing PKCE challenge or a certificate problem. It cannot prove that no account-linking, tenant-isolation, or supply-chain vulnerability exists.

5. Prioritized risk register

These are architecture risks to assess, not confirmed live vulnerabilities.

Priority	Risk	Required mitigation	Owner
P0	PKCE omission/downgrade or code replay	Enforced PKCE and negative redemption tests	Identity
P0	Wrong issuer or school identity mapping	Transaction-bound connection and validated issuer	Identity
P0	Cross-tenant API/file/search/AI access	Server policy and database/storage negative tests	Platform/security
P0	Wrong-tenant custom-domain mapping	Registry uniqueness and activation probes	Platform operations
P0	Account takeover through linking/recovery	Proof-of-control workflows and recovery review	Identity/support
P1	Parent-domain cookie exposure	Host-only sessions and tenant-content separation	Web/security
P1	Dangling domain or stale callback	Controlled removal and fresh re-verification	Platform operations
P1	DNS/ACME credential compromise	Least privilege and key-management controls	Infrastructure
P1	Renewal or validation delegation failure	Monitoring, runbook, first-party fallback access	Operations
P1	Refresh-token replay	Rotation/sender constraint and revocation tests	Identity
P2	Stale audit evidence	Versioned evidence and release freshness gates	Security assurance

6. Phased security-hardening milestones

Quarter labels are relative to kickoff. Dates and staffing follow the initial reconciliation.

Phase 1 — Identity and tenancy spine: Q1–Q2

Owners: identity lead, platform lead, security reviewer, institutional IT.

Deliver:

- Canonical account and external-identity inventory.

- Applicable-control register.
- Maintained OIDC implementation and approved federation connections.
- PKCE, exact redirects, token validation, revocation, recovery, and linking.
- Tenant enforcement across APIs, storage, search, and AI.
- Monitor in reporting-only mode.

Exit gates:

- No unresolved P0 identity or isolation findings.
- All applicable negative tests have current evidence.
- Key rotation, revocation, and recovery drills pass.
- School federation is institution-authorized.
- Exceptions are bounded, approved, and expiring.

The existing project specification already requires evidence-backed lifecycle and authorization enforcement; visual completion is not an activation gate. [7]

Phase 2 — Domains, TLS, and controlled service pilots: Q3

Owners: platform operations, identity, messaging, privacy/support.

Deliver:

- Verified domain registry and lifecycle worker.
- Managed wildcard/per-host certificates.
- Domain and certificate monitors.
- One approved custom-host pilot.
- Invitation-only Mail pilot.
- Exact callback registration tied to domain lifecycle.
- Audit monitor integrated into deployment gates.

Exit gates:

- Wrong-host and wrong-tenant requests fail.
- Renewal failure and domain-removal drills pass.
- Stale DNS/callbacks cannot silently reactivate access.
- Mail restore/export and support controls pass.
- No public activation with unresolved critical findings.

Phase 3 — Multi-school suite and governed expansion: Q4

Owners: product/platform leads and institutional domain owners.

Deliver:

- Multi-school workspace lifecycle.
- Shared Calendar, Files, Action Center, scoped Search.
- Graduation/transfer/offboarding continuity.
- Additional custom domains through the same verified workflow.
- Reviewed partner OIDC pilot.
- Continuous drift findings and remediation tracking.

Exit gates:

- Independent institution boundaries pass end-to-end tests.
- Partner audiences/scopes cannot reach unrelated services.
- Offline caches honor permission expiry under documented behavior.
- Each enabled capability has support, rollback, portability, and authorization evidence.

7. Required release artifact

Each release should produce a security evidence packet containing:

1. Applicable controls and explicit exclusions.
2. Configuration versions and deployment identifiers.
3. Test results and evidence references.
4. Open findings with severity and owner.
5. Approved exceptions and expiry.
6. Institution approvals where required.
7. Domain/certificate inventory.
8. Recovery, revocation, removal, and rollback drill results.
9. Separate labels for designed, implemented, tested, deployed, institution-authorized, and production-enabled.

That gives Semester a defensible assurance process: standards-backed controls, runtime proof, and operational ownership—without presenting a checklist or automated monitor as certification.

⁂

1. https://www.rfc-editor.org/rfc/rfc9700.html
2. https://www.rfc-editor.org/rfc/rfc8555.html
3. https://oauth.net/2/pkce/

4. https://openid.net/specs/openid-connect-core-1_0.html
5. https://vercel.com/docs/domains/working-with-domains/add-a-domain
6. https://vercel.com/docs/domains/troubleshooting
7. The Main Semester design system (2) copy/handoff/specifications/role-system-execution-prompt.pdf
8. https://vercel.com/docs/domains/working-with-nameservers
9. https://vercel.com/docs/domains/managing-dns-records
10. https://www.scalekit.com/blog/oauth-2-0-best-practices-rfc9700
11. https://www.rfc-editor.org/info/rfc8555/
12. https://www.rfc-editor.org/info/rfc9700/
13. https://www.rfc-editor.org/info/rfc10017/
14. https://workos.com/blog/oauth-best-practices
15. https://vercel.com/docs/platforms/multi-tenant-platforms/configuring-domains
16. https://vercel.com/docs/domains/working-with-domains
17. https://github.com/vercel/vercel/discussions/7739
18. https://www.rfc-editor.org/subjects/dns/
19. https://www.rfc-editor.org/info/rfc8657/